

72.39 - Autómatas, teoría de lenguajes y compiladores



Entrega III: Backend

Grupo G-117

14/06/2026

Tabla de contenidos

Tabla de contenidos.....	1
Equipo.....	1
1. Introducción.....	1
2. Modelo computacional.....	2
2.1. Dominio.....	2
2.2. Lenguaje.....	2
3. Implementación.....	5
3.1. Frontend.....	5
3.1.1. Análisis Léxico.....	5
3.1.2. Analisis Sintactico.....	5
3.2. Backend.....	6
3.2.1. Análisis Semántico.....	6
3.2.2. Motor de simulación.....	6
3.2.3. Generación de Código.....	7
3.3. Adicionales.....	8
3.3.1. Casos de Prueba.....	8
3.3.2. Herramienta externa utilizada.....	10
3.4. Dificultades encontradas.....	10
4. Futuras Extensiones.....	11
5. Conclusiones.....	11
6. Referencias.....	11
7. Bibliografía.....	11

Equipo

Nombres	Apellidos	Legajo	Email
Octavio Nicolás	Roberts Sánchez	65861	orobertssanchez@itba.edu.ar
Mariana Lucía	Díaz Noia	65664	mdiaznoia@itba.edu.ar
Joaquín Esteban	Cofiño	62741	jcofino@itba.edu.ar
Amparo Julieta	Oscari Molina	65030	aoscarimolina@itba.edu.ar

1. Introducción

En este informe se documenta el diseño e implementación de Ecompiler, un compilador para un DSL (Domain Specific Language) orientado al modelado y la simulación de ecosistemas biológicos.

El lenguaje permite describir especies, regiones y ecosistemas, poblarlos con individuos, definir comportamientos de interacción entre especies y ejecutar simulaciones evolutivas. A partir de un archivo fuente, el compilador ejecuta la simulación directamente e imprime como artefactos de salida un archivo JSON con el historial completo y un archivo HTML con tablas y un gráfico de curvas de población.

El desarrollo se estructuró en tres etapas: Stage I (diseño del dominio y la sintaxis), Stage II (frontend: análisis léxico y sintáctico con Flex y Bison, construcción del AST) y Stage III (backend: análisis semántico, motor de simulación y generación de archivos de salida).

2. Modelo computacional

2.1. Dominio

El dominio elegido es la biología de ecosistemas: el modelado de poblaciones de especies que coexisten e interactúan dentro de regiones geográficas con condiciones ambientales propias. La motivación surge de la necesidad de simular fenómenos ecológicos (reproducción, extinción, cambios ambientales).

Las entidades centrales del dominio son tres: la especie (conjunto de atributos biológicos compartidos: esperanza de vida, tasa de reproducción, velocidad, dieta, hábitat y tolerancia ambiental), la región (área geográfica con temperatura, humedad, altitud, capacidad de carga y tipo de hábitat) y el ecosistema (agrupación de regiones y especies que conforman el ámbito de la simulación).

Sobre esas entidades el lenguaje permite expresar acciones: agregar individuos a regiones, definir qué ocurre cuando dos especies se encuentran (bloque `on encounter`), programar cambios ambientales en generaciones específicas o aleatorias y disparar la simulación por un número determinado de generaciones. Los comportamientos que no se especifican explícitamente se ignoran por defecto, lo que alivia al programador de describir toda interacción posible.

2.2. Lenguaje

Un programa de Ecompiler es una secuencia no vacía de statements que se ejecutan en el orden en que aparecen, con la excepción de los bloques reactivos (`on encounter`, `on generation`, `every random`), que el motor de simulación invoca durante la ejecución cuando se cumple la condición que les da origen. La sintaxis combina construcciones declarativas, para describir el escenario (especies, regiones, ecosistemas), y construcciones imperativas, para poblarlo, modificarlo y simularlo.

El lenguaje distingue dos familias de tipos. Los tipos primitivos —`int`, `float`, `boolean` y `string`— permiten declarar variables de propósito general (por ejemplo, `int contador = 0;`) y tipan los atributos numéricos de especies y regiones. Los tipos del dominio —`species`, `region` y `ecosystem`— no se declaran como variables sino que surgen de las definiciones `species <id> = createSpecies {...}`, `region <id> {...}` y `ecosystem <id> {...}`; sus identificadores se usan luego para referenciarlas en el resto del programa. Además, existen valores enumerados

en mayúsculas para los aspectos cualitativos del dominio: reproductionStrategy (R_SELECTED/R_STRATEGY y K_SELECTED/K_STRATEGY, tratados como sinónimos), diet (HERBIVORE, CARNIVORE, OMNIVORE, DECOMPOSER) y habitat, que admite distintos valores según se aplique a una especie (TERRESTRIAL, AQUATIC, AMPHIBIOUS) o a una región (TERRESTRIAL, AQUATIC, MIXED).

Una especie se define mediante species <id> = createSpecies { ... }, donde el bloque enumera sus atributos biológicos: lifespan (int), energy (float), reproductionRate (float, en [0.0, 1.0]), speed (int), reproductionStrategy, diet, habitat y un bloque environmentalTolerance que fija, para temperature, humidity y altitude, un rango admisible mediante .atributo = [min, max];. Una región se define con region <id> { temperature = <int>; humidity = <int>; altitude = <int>; carryingCapacity = <int>; habitat = <enum>; }. Un ecosistema agrupa especies y regiones previamente declaradas — ecosystem <id> { region <id>; species <id>; ... } — y debe contener al menos un miembro.

Sobre estas entidades, el lenguaje ofrece primitivas para poblar y modificar el escenario: add N <especie> to <ecosistema>.<región>; agrega N individuos, remove <especie>; elimina un individuo y move <especie> to <ecosistema>.<región>; lo traslada a otra región. Los atributos de un individuo, especie o región se leen y modifican mediante acceso punteado (<id>.<atributo>) y los operadores =, +=, -=, *= y /=; dentro de los bloques on generation y every random, temperature y humidity pueden modificarse sin prefijo, ya que se refieren implícitamente a la región del bloque.

El comportamiento dinámico del ecosistema se programa con tres bloques reactivos. on encounter <A> with in <eco>.<región> { ... } define qué ocurre cuando individuos de las especies A y B coexisten en una región; si no se define, la interacción se ignora. on generation <N> in <eco>.<región> { ... } aplica un cambio en una generación fija, y every random generations in <eco>.<región> { ... } lo hace en generaciones elegidas aleatoriamente, habitualmente combinado con random[min, max] para obtener magnitudes impredecibles. La simulación se dispara con simulate <eco> for N generations; (opcionalmente with seed N para resultados reproducibles).

Para el control de flujo, el lenguaje provee if (<condición>) {...} else {...} —con el dangling else resuelto a favor del if más cercano—, while (<condición>) {...} y for each <especie> in <eco>.<región> {...}, que itera sobre los individuos de una especie presentes en una región. Las expresiones admiten operadores aritméticos (+, -, *, /) con la precedencia matemática habitual, relacionales (<, >, ==, !=, <=, >=) y lógicos (AND, OR, NOT), además de dos construcciones propias del dominio: population of <especie> in <eco>.<región>, que devuelve la cantidad de individuos, y el operador <especie> in <eco>.<región>, que evalúa si al menos un individuo de esa especie está presente en la región.

Finalmente, log population of <especie> in <eco>.<región>;, log state of <eco>; y log generation; permiten inspeccionar durante la ejecución la población de una especie, el estado completo de un ecosistema o el número de generación actual; esta información también queda registrada en los artefactos de salida (ver § 3.2.3). El lenguaje admite comentarios de línea (//...) y de bloque (/ * ... */), descartados por el analizador léxico.

A modo de ejemplo, el siguiente fragmento combina varias de estas construcciones: la definición de una especie y una región, su agrupación en un ecosistema, el agregado de individuos y un bloque on encounter que resuelve un encuentro depredador-presa.

```
species Puma = createSpecies {
    int lifespan = 15;
    float reproductionRate = 0.4;
    int speed = 8;
    reproductionStrategy = reproductionStrategy.K_SELECTED;
    diet = diet.CARNIVORE;
    habitat = habitat.TERRESTRIAL;
    environmentalTolerance;
    .temperature = [5, 35];
    .humidity = [30, 80];
    .altitude = [0, 4000];
}

region Bosque {
    temperature = 20;
    humidity = 70;
    altitude = 800;
    carryingCapacity = 200;
    habitat = TERRESTRIAL;
}

ecosystem Patagonia {
    region Bosque;
    species Puma;
}

add 10 Puma to Patagonia.Bosque;

on encounter Puma with Guanaco in Patagonia.Bosque {
    if (Puma.speed > Guanaco.speed) {
        Guanaco.energy -= 40;
        Puma.energy += 40;
    } else {
        move Guanaco to Patagonia.Bosque;
    }
}
```

Código 1: Ejemplo de un ecosistema con una especie, una región, su agrupación y un encuentro depredador-presa.

3. Implementación

3.1. Frontend

3.1.1. Análisis Léxico

Para la implementación del analizador léxico se utilizó la herramienta Flex. Se definieron reglas léxicas capaces de reconocer los distintos elementos del lenguaje, incluyendo palabras reservadas asociadas al dominio de simulación (como especies, regiones y ecosistemas), identificadores definidos por el usuario, constantes enteras, reales, booleanas y cadenas de texto. Asimismo se reconocen operadores aritméticos, relacionales y lógicos, además de los símbolos de puntuación utilizados para estructurar los programas del lenguaje.

Se implementó soporte para comentarios multilínea mediante las funciones `EnterMultilineCommentLexemeAction()` y `LeaveMultilineCommentLexemeAction()`, permitiendo ignorar fragmentos de código que no forman parte de la simulación. Asimismo, los espacios en blanco y demás secuencias irrelevantes son procesados mediante `IgnoredLexemeAction()`, evitando que lleguen al parser.

Finalmente, el analizador léxico incorpora mecanismos básicos de detección de errores. Al detectar una secuencia de caracteres que no coincide con ninguna de las reglas definidas para el lenguaje, se genera un token de error y se interrumpe el proceso de compilación, permitiendo informar al usuario la existencia de símbolos inválidos en el programa fuente.

3.1.2. Analisis Sintactico

El análisis sintáctico fue implementado con Bison en modo LALR(1). La gramática tiene como símbolo inicial `program`, definido como una lista no vacía de `statements`. Un `statement` puede ser una definición de especie, región o ecosistema, una acción sobre poblaciones (`add`, `remove`, `move`), un bloque de comportamiento (`on encounter`, `on generation`, `every random`), una sentencia de simulación, una estructura de control (`if/else`, `while`, `for each`), una instrucción de log o una asignación de atributo.

La precedencia y asociatividad de los operadores se declaró mediante las directivas `%left` y `%right` de Bison. Los operadores lógicos `OR` y `AND` tienen menor precedencia que los relacionales; `NOT` tiene asociatividad derecha; los relacionales son de mayor precedencia que los aritméticos, y entre los aritméticos, la suma y resta tienen menor precedencia que la multiplicación y la división, respetando la convención matemática estándar.

El conflicto clásico del `if/else` pendiente (`dangling else`) se resolvió con la directiva `%prec "then"` aplicada a la producción del `if` sin rama `else`, asignándole menor precedencia que el token `ELSE`, de modo que el parser desplace el `else` en lugar de reducir prematuramente.

Para cada nodo del AST existe una función `SemanticAction` en `BisonActions.c` que aloca la estructura correspondiente, copia los strings con `strdup` (liberando el puntero original que Flex aloca dinámicamente) y la encadena en la lista de `statements`. `AbstractSyntaxTree.h` define las estructuras de datos de todos los nodos y `AbstractSyntaxTree.c` implementa las funciones de destrucción recursiva que liberan la memoria del árbol al finalizar la compilación.

3.2. Backend

3.2.1. Análisis Semántico

El análisis semántico se implementa en `SemanticAnalyzer.c` y opera sobre el AST construido por el frontend. Se realiza en tres pasadas secuenciales sobre la lista de `statements`, lo que permite que las definiciones de especies, regiones y ecosistemas se registren en la tabla de símbolos antes de validar las sentencias que las referencian.

La primera pasada registra todas las definiciones de `species` y `region`. Durante el registro se validan invariantes de dominio: la tasa de reproducción de una especie debe estar en el rango $[0.0, 1.0]$; la temperatura de una región debe estar entre -50 y 60 °C; la humedad entre 0 y 100 ; la altitud debe ser no negativa. También se detectan definiciones duplicadas: si una especie o región ya existe en la tabla de símbolos con el mismo nombre, se reporta un error y la compilación falla.

La segunda pasada registra los ecosistemas y verifica que cada uno de sus miembros (`species` y `region` declarados dentro del bloque `ecosystem`) esté ya definido en la tabla. Un ecosistema vacío también es rechazado.

La tercera pasada valida las sentencias de uso: los `statements add` verifican que la especie, el ecosistema y la región existan en la tabla; la detección de tipos incompatibles en expresiones binarias (por ejemplo, sumar un entero y un string) se realiza mediante la función `_inferExpressionType`. Los siete casos de rechazo cubiertos, correspondientes a los casos de prueba de rechazo del Stage I, son:

- I. región no definida en `add`,
- II. especie no definida en `add`,
- III. parámetros de región fuera de rango,
- IV. tipo de dato incompatible en expresión binaria,
- V. ecosistema vacío,
- VI. `reproductionRate` fuera del rango $[0.0, 1.0]$,
- VII. nombre de región duplicado.

Más casos de rechazo fueron añadidos posteriormente.

La tabla de símbolos es una estructura `SymbolTable` (`SemanticAnalyzer.h`) con un arreglo dinámico de `SymbolEntry` que crece por duplicación de capacidad. Cada entrada almacena el nombre, el tipo de símbolo (`SYMBOL_SPECIES`, `SYMBOL_REGION` o `SYMBOL_ECOSYSTEM`) y un puntero al nodo del AST correspondiente para uso potencial en fases posteriores. La tabla se destruye al finalizar la compilación.

3.2.2. Motor de simulación

El motor de simulación (`SimulationEngine.c`) es la fase de dominio del backend. Lee el AST como entrada de solo lectura y opera sobre estructuras de runtime independientes definidas en `SimulationState.h`: `Individual` (individuo con nombre de especie, energía y edad), `RuntimeSpecies` (atributos compartidos de la especie en runtime), `RuntimeRegion` (región con condiciones ambientales actuales y lista enlazada de individuos) y `RuntimeEcosystem` (agrupación de `RuntimeRegion` y `RuntimeSpecies` con su propio nombre). `SimulationState` coordina todas estas estructuras junto con los historiales de población, extinción, encuentros y cambios ambientales.

La inicialización (`initSimulation`) recorre el AST buscando definiciones de ecosistema e instancia las estructuras de runtime construyendo `RuntimeSpecies` desde `SpeciesDefinition` y `RuntimeRegion` desde `RegionDefinition` mediante las funciones `_buildSpecies` y `_buildRegion` respectivamente. La referencia al AST se guarda como `programStatements` en `SimulationState` para ser consultada durante la simulación.

La ejecución (`runSimulation`) comienza con una fase de pre-simulación que ejecuta todos los `statements` `add`, `remove` y `move` que aparecen en el programa fuente antes del bloque `simulate`. Esto separa la fase de configuración (poblar regiones) de la fase de simulación propiamente dicha. A continuación, por cada generación y por cada región del ecosistema, el motor aplica cinco pasos en orden: (1) `_applyOnGeneration`, que activa los bloques `on generation` cuya generación coincide con la actual; (2) `_applyEveryRandom`, que activa los bloques `every random` con probabilidad 50 %; (3) `_applyEncounters`, que para cada individuo de la especie A busca un individuo aleatorio de la especie B y ejecuta el cuerpo del bloque `on encounter` con ambos individuos en contexto; (4) `_applyReproduction`, que aplica la tasa de reproducción de cada especie para eventualmente agregar un nuevo individuo si hay capacidad; (5) `_applyMortality`, que envejece cada individuo y lo elimina si supera su `lifespan`, pierde toda la energía o se encuentra en un hábitat incompatible (con probabilidad 50 % por generación).

El contexto de ejecución (`_ctxA` y `_ctxB`) es un par de estructuras `ExecCtx` que apuntan al individuo activo en cada rol durante un bloque `on encounter` o `for each`. Las funciones de evaluación de expresiones (`_evalExpr`) y condiciones (`_evalCond`) consultan este contexto para acceder a los atributos `mutable energy` del individuo y a los atributos compartidos `speed`, `lifespan` y `reproductionRate` de la especie.

Al finalizar cada generación se registran `snapshots` de población (`PopulationRecord`), ambiente (`EnvironmentRecord`) y, cuando corresponde, extinción (`ExtinctionRecord`) y encuentro (`EncounterRecord`) en listas enlazadas con `tail pointer` para inserción en $O(1)$. Estos historiales son consumidos posteriormente por los escritores de JSON y HTML.

3.2.3. Generación de Código

El compilador construye una representación interna del escenario descrito por el usuario y ejecuta la simulación solicitada. En esta última etapa se realiza la generación de dos artefactos de salida (`simulation.html` y `simulation.json`) a partir de la representación interna obtenida luego del análisis y ejecución de la simulación.

En primer lugar, el archivo `simulation.json` constituye la representación estructurada de los resultados de la simulación. En él se almacena información sobre la evolución del sistema a lo largo de las generaciones (poblaciones registradas por especie, cambios ambientales ocurridos, encuentros entre especies, eventos de extinción detectados). Este formato fue elegido por su simplicidad, legibilidad y facilidad de procesamiento por herramientas externas.

{

```

"generationCount": N,
"ecosystems": [
  ...
],
"populationHistory": [
  ...
],
"environmentHistory": [
  ...
],
"encounters": [
  ...
],
"extinctions": [
  ...
]
}

```

Código 2: Estructura de salida en archivos JSON

En el **Cód. 2** se ve la estructura establecida para los artefactos de salida de tipo .json, donde en cada matriz se incluye información relevante de la simulación por generación.

Por otro lado, el archivo simulation.html presenta los resultados de la simulación mediante una interfaz visual compuesta por tablas y gráficos. Entre ellos se incluyen curvas de aptitud para la supervivencia de las especies en función de la cantidad de generaciones simuladas. La información utilizada para construir estas visualizaciones se obtiene directamente de las estructuras generadas durante la ejecución de la simulación, lo que garantiza consistencia con los resultados almacenados en el archivo JSON.

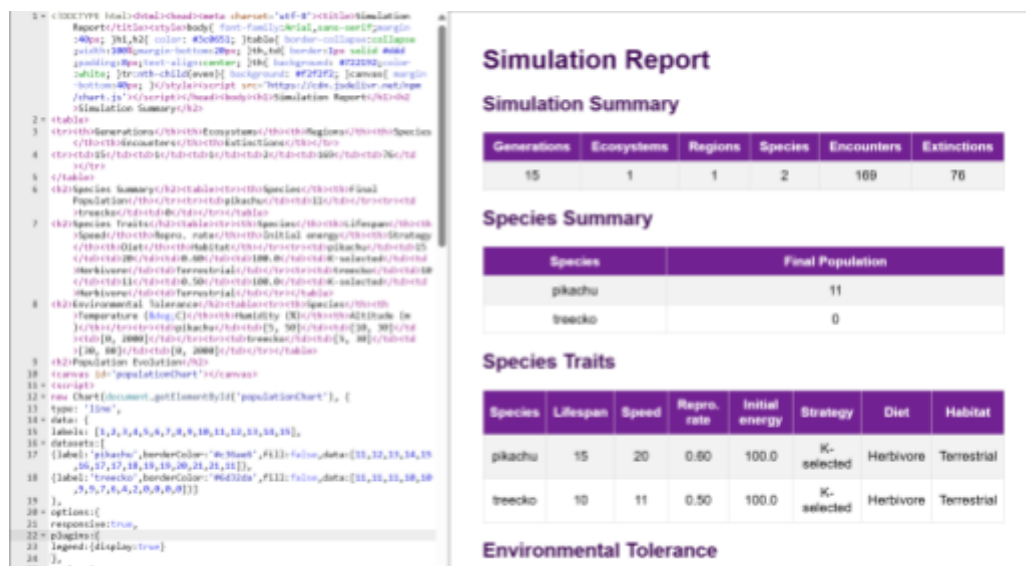


Imagen 1: Vista parcial del simulation.html a través de un visor de HTML (test .../accept/25-battle)

Population Evolution

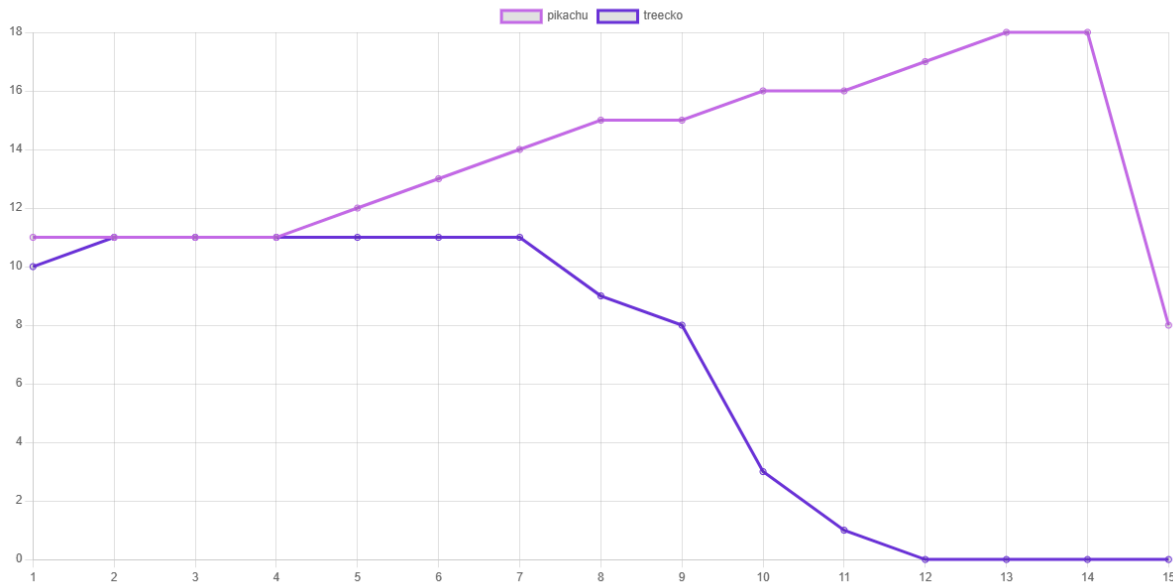


Imagen 2: Visualización de uno de los gráficos disponibles del `simulation.html` a través de un visor de HTML (test `.../accept/26-another-battle`)

De esta forma, el archivo JSON proporciona una representación persistente y fácilmente intercambiable de los datos de la simulación, mientras que el archivo HTML ofrece una presentación orientada al análisis e interpretación de dichos resultados.

3.3. Adicionales

3.3.1. Casos de Prueba

El proyecto cuenta con dos tipos de casos de testeo: los casos que el compilador debe aceptar (carpeta `accept/`) y los que debe rechazar (carpeta `reject/`).

Los casos de aceptación comprenden:

- ecosistema básico con una región y una especie (`accept/01`)
- bloque `on encounter` con primitivas de modificación de atributos (`accept/02`)
- cambio ambiental fijo con `on generation` (`accept/03`)
- iteración con `for each` (`accept/04`)
- coexistencia sin interacción definida (`accept/05`)
- operador `in` en condicional (`accept/06`)
- especie en hábitat incompatible con mayor mortalidad (`accept/07`)
- `log state` del ecosistema (`accept/08`)
- definición completa de especie (`accept/09`)
- definición completa de región (`accept/10`)
- agregado de individuos (`accept/11`)
- comentarios de línea (`accept/12`)

- comentarios de bloque (accept/13)
- parámetros de `region` en los límites exactos del rango válido (accept/14)
- `reproductionRate` en los límites exactos 0.0 y 1.0 (accept/15)
- bucle `while` sobre la población de una especie (accept/16)
- simulación con semilla explícita (`with seed`) (accept/17)
- ecosistema con varias regiones y población repartida (accept/18)
- evento ambiental aleatorio con `every random generations` (accept/19)
- comportamientos predefinidos: `hunt`, `flee`, `eat`, `drink`, `harvest`, `ignore` (accept/20)
- declaración de variables de los cuatro tipos primitivos (accept/21)
- especie anfibia (`AMPHIBIOUS`) en región de hábitat mixto (`MIXED`) (accept/22)
- condiciones compuestas con operadores lógicos `AND`, `OR` y `NOT` (accept/23)
- números negativos en valores de atributos, rangos y expresiones (accept/24)
- pelea entre dos pokemones: `Houndoom` y `Scizor` (accept/25)
- otra pelea entre dos pokemones: `Pikachu` y `Treecko` (accept/26)

Los casos de rechazo cubren:

- región no definida en `add` (reject/01)
- especie no definida en `add` (reject/02)
- parámetros de región fuera de rango físico (temperatura 2000, humedad 150, altitud 100000) (reject/03)
- operación aritmética entre tipos incompatibles (reject/04)
- ecosistema vacío (reject/05)
- `reproductionRate` fuera de `[0.0, 1.0]` (reject/06)
- nombres de región duplicados (reject/07)
- `reproductionRate` por encima de 1.0, con sintaxis válida (reject/08)
- temperatura por encima del máximo (200) (reject/09)
- humedad por encima del máximo (150) (reject/10)
- especies con nombre duplicado (reject/11)
- especie no definida referenciada dentro del ecosistema (reject/12)
- especie no definida en `add`, con sintaxis válida (reject/13)
- punto y coma faltante al final de una sentencia (reject/14)
- `region` con campos obligatorios faltantes (reject/15)
- carácter fuera del alfabeto del lenguaje (reject/16)
- programa vacío (reject/17)
- altitud negativa (-100) (reject/18)
- temperatura por debajo del mínimo (-60) (reject/19)
- `reproductionRate` negativo (-0.3) (reject/20)

3.3.2. Herramienta externa utilizada

Es importante mencionar el uso de la librería [chart.js](#) en el `src/main/c/code-generation/html/HtmlWriter.c` para la construcción de gráficos que describen

el desarrollo de la ejecución de un script particular. Si bien no se pidió permiso explícito de la cátedra, se utilizó esta herramienta y se documentó dentro de README.

3.4. Dificultades encontradas

Enfrentamos cierta dificultad en la etapa de generación de código, ya que en el Stage 1 se planteó generar el archivo `simulation.html` a partir de la información almacenada en `simulation.json`. Este enfoque permitía utilizar el JSON como fuente única de datos.

Durante la implementación, se observó que esta estrategia implicaba serializar la información de las estructuras internas a JSON para luego volver a procesar inmediatamente con el único objetivo de construir el reporte HTML. Dado que toda la información necesaria ya se encontraba disponible en memoria, esta solución introducía una etapa adicional de parseo y reconstrucción de estructuras sin aportar beneficios significativos.

Como consecuencia, se decidió generar tanto `simulation.json` como `simulation.html` directamente a partir de las estructuras internas construidas durante la ejecución de la simulación. Esta modificación simplificó la implementación y permitió acceder a información agregada, útil para la construcción de tablas y gráficos estadísticos.

Otra problemática surgió tras la evaluación de algunos casos de testeo donde se debía implementar variables con números negativos, donde se descubrió que `integer` y `float` estaban definidos sin signo porque el `'-'` siempre era `SUB_OP`. Un `'altitude = -100'` se tokenizaba como `'... ASSIGN SUB_OP INTEGER'` y rompía la regla en el parser antes de llegar a la semántica. Una solución evaluada fue definir, en el caso de `INTEGER`, la regla como `'-?{digit}+'`, pero esta solución rompía la resta binaria porque `'... poblacion - 5'` pasaba a leerse como `'poblacion (-5)'` sin un operador entre los dos valores.

Como solución a este problema, se añadió soporte de números negativos en la gramática y no en el lexer para no romper la resta binaria, mediante el uso de `signedInteger` y `signedFloat`, y el menos unario, lo que permite a Bison distinguir entre un `'-'` unario al inicio de una expresión y la resta binaria entre dos expresiones. Este cambio es necesario para la asignación de valores negativos a los campos como `temperature`, y a variables sueltas (sin restricciones). Este cambio no rompe las restricciones de ciertos campos en donde no puede existir valores negativos, pues se valida de forma semántica. El caso de validación de usos negativos está en el test *accept/24-negative-values* donde se le asigna a una variable cualquiera un valor negativo.

4. Futuras Extensiones

Una posible extensión consiste en enriquecer el modelo de simulación incorporando nuevas interacciones entre especies, tales como relaciones simbióticas, competencia por recursos y mecanismos de cooperación. Esto permitiría representar ecosistemas más complejos y realistas.

También, como extensión útil, se podría desarrollar un método de testeo de los archivos resultantes `.json` y `.html`, mediante herramientas externas, para verificar que estén bien formados.

5. Conclusiones

El desarrollo de Ecompiler permitió ver la construcción de un lenguaje de dominio específico. Al modelar un ecosistema biológico como dominio, el lenguaje logró que nos concentremos exclusivamente en la descripción declarativa del sistema, delegando al compilador la responsabilidad de resolver la dinámica subyacente.

El recorrido por las cuatro fases de compilación puso en evidencia que los conceptos teóricos desarrollados a lo largo de la materia no son abstracciones aisladas sino herramientas que se utilizan para la construcción de software real. La teoría de autómatas, las gramáticas libres de contexto y la semántica atribuida adquirieron significado concreto al enfrentarse a decisiones de diseño con consecuencias directas sobre el comportamiento del compilador.

6. Referencias

1. Cátedra de Autómatas, Teoría de Lenguajes y Compiladores. Flex-Bison-Compiler. Disponible en: <https://github.com/agustin-golmar/Flex-Bison-Compiler>.
2. The Flex Project. Flex: The Fast Lexical Analyzer Generator, Manual. Disponible en: <https://westes.github.io/flex/manual/>
3. Free Software Foundation. Bison: GNU Parser Generator, Manual. Disponible en: <https://www.gnu.org/software/bison/manual/>
4. Chart.js : <https://www.chartjs.org/docs/latest/>

7. Bibliografía

1. [\(2026-03-12\) Proyecto Especial](#).
<https://docs.google.com/document/d/1XZy63UwUVHyMCR6p051aer-i9SpbLmoOSGC-gh6Cb-I/edit?tab=t.0#heading=h.detc7r7352wx>
2. [\(2026-03-12\) Modelo Computacional](#).
https://docs.google.com/document/d/1V6DsOd9hhpiCoMlqdqVNyaVvWomIZ8ylNW4TD_rruDY/edit?tab=t.0#heading=h.detc7r7352wx
3. [\(2025-09-03, v1.0.0\) Análisis Léxico](#).
<https://docs.google.com/document/d/1o0rYT3vbLtdxuMNnIirf7je2bp2B6FuPGiwCJkVrx0g/edit?tab=t.0#heading=h.detc7r7352wx>
4. [\(2024-05-08, v0.1.0\) Análisis Sintáctico](#).
https://docs.google.com/document/d/1HNNtofu_TCHadDxham2K-_HwKfaCG55izvX8a-QtqZU/edit?tab=t.0#heading=h.detc7r7352wx
5. [\(2024-09-25, v0.2.0\) Análisis Semántico](#).
https://docs.google.com/document/d/1ZqUt72POtTA-IXPHUVgFqz0hbb7-QQnc5i_Q0B1rMPY/edit?tab=t.0#heading=h.detc7r7352wx
6. [\(2024-05-16, v0.1.0\) Generación de Código](#).
https://docs.google.com/document/d/1gTDatettvr3GLewUy4_iwacQvTCZAAks02xu6l6eFcc/edit?tab=t.0#heading=h.detc7r7352wx